

Complementos de Investigação Operacional

3.1 Problemas sequenciais: regra de Johnson (2 máquinas)

José Firmino Aguilar Madeira

Porquê sequenciar?

Uma **gráfica** recebe trabalhos que passam todos pelas mesmas duas máquinas, **pela mesma ordem**: primeiro a **impressão**, depois o **acabamento** (corte, dobragem, encadernação).

Por que ordem se devem fazer os trabalhos?

Objetivo: minimizar o **tempo total** (*makespan*) T — o instante em que o último trabalho sai do acabamento. Menos $T \Rightarrow$ menos tempo ocioso, menos horas extraordinárias, entregas mais cedo.

O mesmo problema aparece em oficinas (maquinar $\rightarrow$ montar), laboratórios (preparar $\rightarrow$ analisar) e *data centers* (ler $\rightarrow$ processar).

Hipóteses

- tempos de processamento conhecidos;
- cada máquina faz um trabalho de cada vez;
- um trabalho iniciado não é interrompido;
- todos os trabalhos disponíveis no instante 0;
- a mesma sequência nas duas máquinas.

Quantas sequências? $n! \quad 6! = 720$,
 $10! \approx 3,6 \times 10^6$, $20! \approx 2,4 \times 10^{18}$.

O exemplo deste capítulo: seis trabalhos na gráfica

Trabalho		Impressão (h)	Acabamento (h)
A	Catálogos	8	9
B	Revistas	11	5
C	Cartões de visita	5	2
D	Convites	3	5
E	Livros	4	12
F	Cartazes	10	9

Soma: 41 h de impressão e 42 h de acabamento.

Primeira ideia: fazer pela ordem de chegada, A, B, C, D, E, F.

O exemplo deste capítulo: seis trabalhos na gráfica

Trabalho		Impressão (h)	Acabamento (h)
A	Catálogos	8	9
B	Revistas	11	5
C	Cartões de visita	5	2
D	Convites	3	5
E	Livros	4	12
F	Cartazes	10	9

Soma: 41 h de impressão e 42 h de acabamento.

Primeira ideia: fazer pela ordem de chegada, A, B, C, D, E, F.

Como se calcula T ? Para cada trabalho, por ordem:

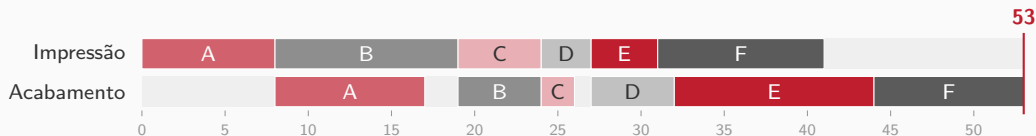
$$C_1 \leftarrow C_1 + a_j, \quad C_2 \leftarrow \max(C_2, C_1) + b_j$$

(a_j, b_j : tempos na impressão e no acabamento;
 C_1, C_2 : instantes em que cada máquina fica livre).

O acabamento só começa um trabalho quando ele saiu da impressão e o anterior saiu do acabamento.

Pela ordem de chegada: $T = 53$ h

Trabalho	A	B	C	D	E	F
Sai da impressão (C_1)	8	19	24	27	31	41
Sai do acabamento (C_2)	17	24	26	32	44	53



O acabamento fica **à espera** no início (8 h, enquanto A é impresso) e entre trabalhos (B sai às 19 h, mas o acabamento ficou livre às 17 h). No total, $53 - 42 = 11$ h de acabamento parado; a impressão termina às 41 h e fica 12 h parada até ao fim.

Pergunta: há uma ordem melhor? Qual é a melhor das 720?

A regra de Johnson (1954)

Algoritmo (2 máquinas)

1. Procurar o **menor** de todos os tempos ainda por atribuir.
2. Se for na **máquina 1**, o trabalho vai para a **primeira** posição livre.
Se for na **máquina 2**, vai para a **última** posição livre.
3. Riscar esse trabalho e voltar a 1.

Empates: qualquer escolha dá uma sequência ótima.

Forma equivalente: primeiro os trabalhos com $a_j \leq b_j$, por ordem crescente de a_j ; depois os trabalhos com $a_j > b_j$, por ordem decrescente de b_j .

Intuição

- **Começar** com impressão curta: o acabamento começa cedo, não fica à espera.
- **Acabar** com acabamento curto: depois de a impressão terminar, falta pouco.

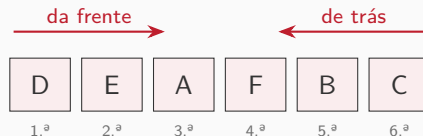
Resultado

A regra de Johnson dá sempre uma sequência **ótima** (com 2 máquinas).

Johnson passo a passo

Passo	Menor tempo	Máquina	Trabalho	Posição
1	2	acabamento	C	6. ^a (última)
2	3	impressão	D	1. ^a
3	4	impressão	E	2. ^a
4	5	acabamento	B	5. ^a
5	8	impressão	A	3. ^a
6	—	—	F	4. ^a

No passo 4, o 5 de D no acabamento já não conta (D já foi atribuído): o menor por atribuir é o 5 de B. No passo 5 restam A (8, 9) e F (10, 9): o menor é o 8 de A, na impressão; F fica com a única posição livre.

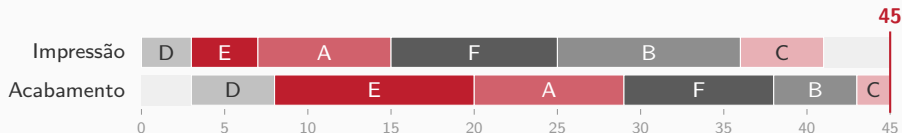


Sequência de Johnson: D, E, A, F, B, C.

Pela forma equivalente: $a \leq b$: D (3), E (4), A (8) por a crescente; $a > b$: F (9), B (5), C (2) por b decrescente.

Com Johnson: $T = 45$ h (menos 8 h)

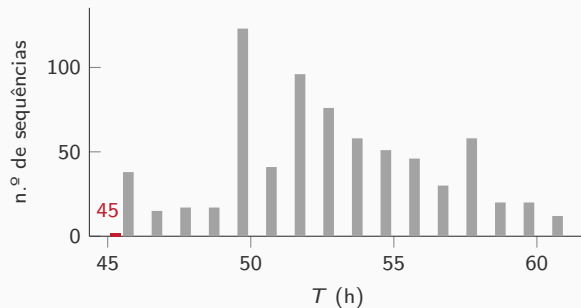
Trabalho	D	E	A	F	B	C
Sai da impressão (C_1)	3	7	15	25	36	41
Sai do acabamento (C_2)	8	20	29	38	43	45



O acabamento só espera 3 h (no início) e nunca mais pára: $45 - 42 = 3$ h ocioso. A impressão termina às 41 h e fica 4 h ociosa.

Ganho: de 53 h para 45 h, -15% , sem comprar máquinas nem contratar ninguém — só mudando a ordem.

Johnson é mesmo ótimo? Enumerar as 720 seqüências



- O mínimo é 45 h: só 2 das 720 seqüências o atingem (D E A F B C e D E F A B C).
- A média é 53 h — a ordem de chegada (53 h) é uma seqüência “típica”.
- A pior dá 61 h.
- As duas ótimas diferem só na ordem de A e F; Johnson encontra uma delas.

Com 6 trabalhos ainda se enumera; com 20 já não. Johnson resolve com n passos, qualquer que seja n .

```
def johnson(a, b):  
    """a, b: dicionários trabalho -> tempo."""  
    L = [j for j in a if a[j] <= b[j]]  
    R = [j for j in a if a[j] > b[j]]  
    L.sort(key=lambda j: a[j])    # a crescente  
    R.sort(key=lambda j: -b[j])   # b decrescente  
    return L + R  
  
def tempo_total(seq, a, b):  
    c1 = c2 = 0  
    for j in seq:  
        c1 += a[j]  
        c2 = max(c2, c1) + b[j]  
    return c2  
  
a = dict(A=8, B=11, C=5, D=3, E=4, F=10)  
b = dict(A=9, B=5, C=2, D=5, E=12, F=9)  
s = johnson(a, b)  
print(s, tempo_total(s, a, b))  
# ['D', 'E', 'A', 'F', 'B', 'C'] 45
```

Verificar com força bruta:

```
from itertools import permutations  
min(tempo_total(p, a, b)  
     for p in permutations(a))  
# 45
```

Ficheiro completo (Johnson para duas máquinas e força bruta):

code/python/03_scheduling/03_1_johnson/
sequenciamento.py

Exemplo 1 — nove trabalhos, duas máquinas

Trabalho	J1	J2	J3	J4	J5	J6	J7	J8	J9
M1 (h)	2	5	4	9	6	8	7	5	4
M2 (h)	6	8	7	4	3	9	3	8	11

Determine a sequência ótima, o tempo total e o tempo ocioso de cada máquina. (Volta em 3.3, com o TDM.)

Exemplo 2 — sete trabalhos, duas máquinas

Trabalho	1	2	3	4	5	6	7
M1 (h)	3	12	15	6	10	11	9
M2 (h)	8	10	10	6	12	1	3

Determine a sequência ótima, o tempo total e o tempo ocioso de cada máquina. Há empates? Mudam o tempo total?

O QUE DEVE FICAR — 3.1

1. Problema: n trabalhos, 2 máquinas, mesma ordem; minimizar o tempo total T . Há $n!$ sequências.
2. Tempos de conclusão: $C_1 \leftarrow C_1 + a_j$, $C_2 \leftarrow \max(C_2, C_1) + b_j$. Tempo ocioso de cada máquina = T – soma dos seus tempos.
3. **Johnson**: o menor tempo por atribuir; na máquina 1 $\rightarrow$ primeiro lugar livre; na máquina 2 $\rightarrow$ último lugar livre.
4. Com 2 máquinas, Johnson é **ótimo**. No exemplo: 53 h $\rightarrow$ 45 h só mudando a ordem.
5. Verificar sempre: recalcular T da sequência obtida (e, com poucos trabalhos, comparar com a força bruta).

A seguir: 3.2 Três ou mais máquinas — quando é que Johnson ainda funciona?

Referências Lista completa: *Bibliografia da UC*.

- Johnson, S. M., Optimal two- and three-stage production schedules with setup times included, *Naval Research Logistics Quarterly*, 1(1), 61–68, 1954.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — cap. 10, Sequencing Problems.
- Taha, H. A., *Operations Research: An Introduction*, 10.^a ed., Pearson, 2017.
- Pinedo, M. L., *Scheduling: Theory, Algorithms, and Systems*, 5.^a ed., Springer, 2016 — *flow shops*.

Recursos

- Lista de exercícios 3. `code/python/03_scheduling/03_1_johnson/sequenciamento.py`.